Generalized Additive Models

ECON 8310: Business Forecasting — Lecture 3

Department of Economics

University of Nebraska at Omaha

October 19, 2026

Lecture Outline

1. Motivation: Beyond Linear, Without Black Boxes
2. GAM Structure and Smoothing
3. Prophet for Time-Series GAMs
4. pyGAM for Generalized Additive Modeling
5. Key Takeaways and Roadmap

Motivation: Beyond Linear, Without Black Boxes

We want flexibility to capture nonlinear relationships while still being able to *explain* what the model is doing.

The Flexibility–Interpretability Tradeoff

Linear models (ARIMA, regression):

- ✓ Each coefficient has a clear meaning
- ✓ Stable, well understood
- × One constant slope everywhere
- × Miss curves and thresholds

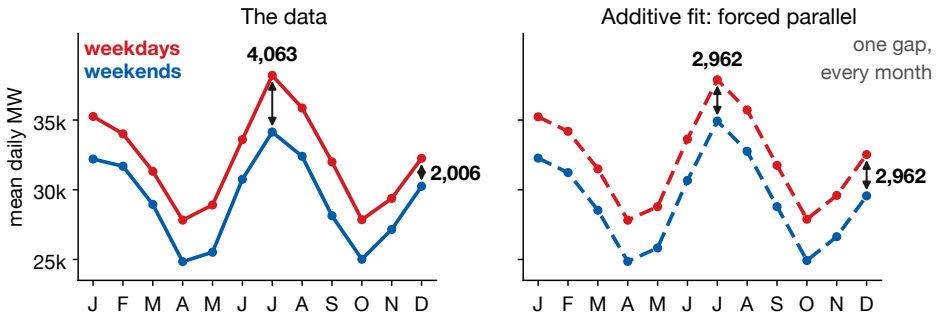
Fully nonlinear models (trees, neural nets):

- ✓ Capture almost any pattern
- ✓ Accurate on complex data
- × Hard to explain one prediction
- × Interactions entangle variables

GAMs sit between: a flexible curve per variable, **added** together, so each effect still plots on its own.

Why Additivity Matters for Interpretation

In an additive model, each variable's effect is the same whatever the others are doing. Lab 3's daily electricity load (PJM East), by month:



An additive model, month + weekend, can only draw parallel curves: one weekend effect for every month. The real gap is twice as big in July as in December. Parallel is the price, and also the gift: each effect becomes one curve you can hand to a stakeholder.

GAM Structure and Smoothing

Additive nonlinear functions, penalized for wiggliness.

What Is a GAM?

Generalized Additive Model

$$y = \alpha + f_1(x_1) + f_2(x_2) + \cdots + f_k(x_k) + \varepsilon$$

where each $f_j(\cdot)$ is a **smooth, nonlinear function** estimated from the data (Wood 2017).

Each f_j is free to be linear, curved, U-shaped, or threshold-like — the data decides. Because the terms are *added*, each one can be plotted on its own. A smoothness penalty, two slides on, keeps them from chasing noise.

A retail example. Weekly sales as $f_1(\text{time})$ for the trend, plus $f_2(\text{week of year})$ for seasonality, plus $f_3(\text{unemployment})$ for economic sensitivity: three curves, each read on its own.

What the "Generalized" Actually Means

So far every y we have modeled has been continuous and roughly symmetric — sales in dollars, demand in units. For that case an *additive* model is enough. The **G** in GAM is what happens when y is not that kind of variable.

Generalized Additive Model, in full

$$g\big(\mathbb{E}[y]\big) = \alpha + f_1(x_1) + \cdots + f_k(x_k)$$

The additive predictor models a *transformation* of the mean. That transformation $g(\cdot)$ is the **link function**.

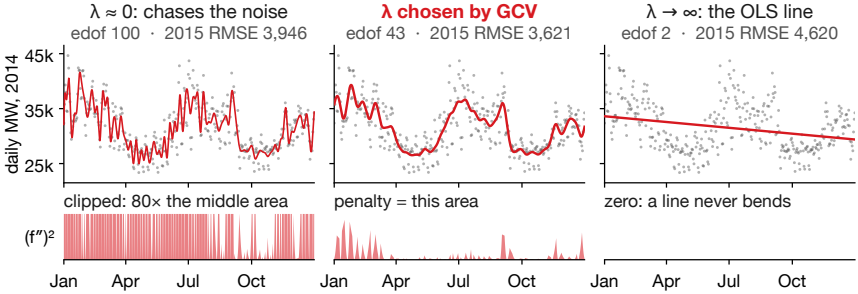
If y is...	**Distribution**	**Link $g(\cdot)$**
Continuous, symmetric (sales)	Gaussian	Identity — $g(\mu) = \mu$
A count (units, visits, claims)	Poisson	Log — keeps the forecast positive
Binary (churn, stockout, yes/no)	Binomial	Logit — keeps it inside $[0, 1]$

In `pyGAM` this is the difference between `LinearGAM`, `PoissonGAM`, and `LogisticGAM` — same additive structure, different link. Everything else in this lecture is unchanged.

Controlling Smoothness: The Penalty Term

A GAM minimizes a **penalized** sum of squared errors:

$$\text{PSSE} = \underbrace{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}_{\text{fit}} + \underbrace{\lambda \int (f''(x))^2 \, dx}_{\text{smoothness penalty}}$$



f'' measures bending, so the penalty is the shaded area: zero only for a straight line. Same data, same 100 splines (cubic pieces joined at knots); only λ changes, and GCV chooses it, not your eye.

Prophet for Time-Series GAMs

Facebook's Prophet decomposes a time series into trend, seasonality, and holiday effects using a GAM.

Prophet: Forecasting at Scale

Taylor and Letham (2018) built Prophet for large-scale forecasting at Facebook (now Meta). It is a GAM in time-series clothing — the series is decomposed additively:

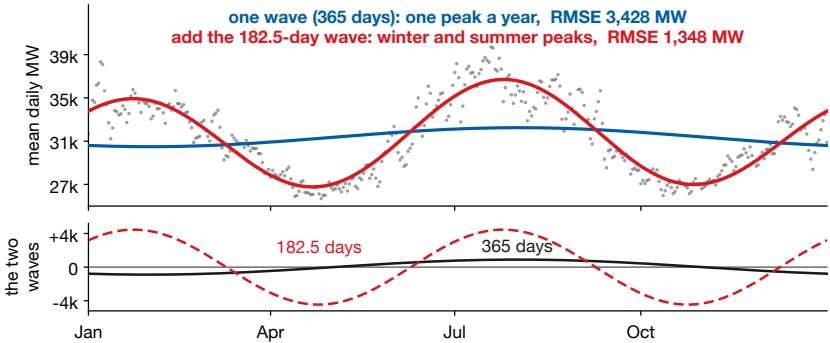
$$y(t) = \underbrace{g(t)}_{\text{trend}} + \underbrace{s(t)}_{\text{seasonality}} + \underbrace{h(t)}_{\text{holidays}} + \varepsilon_t$$

Trend $g(t)$: piecewise linear or logistic, with *changepoints* where the slope may shift. **Seasonality** $s(t)$: a sum of sine waves (a Fourier series), so weekly and yearly cycles fit at once. **Holidays** $h(t)$: irregular events you supply — Black Friday, a launch, a lockdown.

Built for analysts, not statisticians: one fit-and-forecast call, robust to gaps and outliers, and every component plots on its own.

Prophet's Seasonality: How Many Waves?

$s(t)$ is a sum of sine waves. Lab 3's average load for each day of the year, fitted with one wave and then with two:

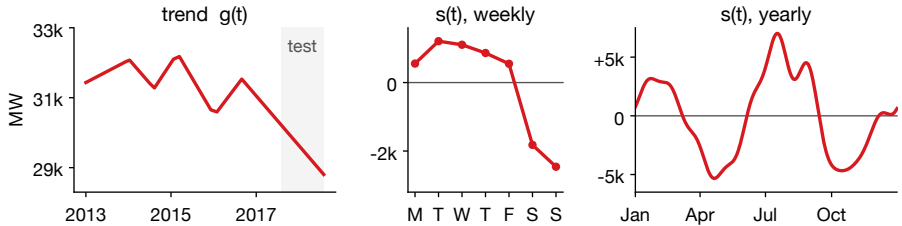


A 365-day wave has one peak a year. Heating in January and cooling in July need the 182.5-day wave too, and adding it cuts the error by a factor of 2.5. Prophet's yearly term uses 10 such pairs by default.

Prophet: Code and Workflow

Prophet on Lab 3's Electricity Data

```
from prophet import Prophet
m = Prophet(yearly_seasonality=True, weekly_seasonality=True, daily_seasonality=False)
m.fit(train)    # a DataFrame with columns ds and y
fc = m.predict(test);  m.plot_components(fc)
```



`plot_components` draws one panel per additive term. **Reach for Prophet** when someone will ask you to explain the forecast; **reach for pyGAM** to see the shape of a few non-calendar drivers.

pyGAM for Generalized Additive Modeling

When you have multiple predictors and want flexible nonlinear effects for each.

pyGAM: Flexible Additive Modeling

`pyGAM` fits a GAM with a smooth function you specify per predictor:

pyGAM Example

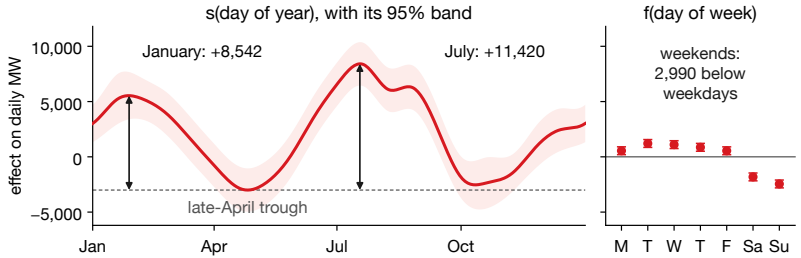
```
from pygam import LinearGAM, s, f, l
gam = LinearGAM(s(0) + s(1) + s(2) + f(3))
gam = gam.gridsearch(X, y)
pdep, confi = gam.partial_dependence(term=0, width=0.95)
y_hat = gam.predict(X_new)
```

The three term types are the whole vocabulary: `s(j)` is a smooth spline of feature j — the flexible nonlinear effect; `l(j)` forces a straight line, which is worth using when theory says the effect should be linear; and `f(j)` handles a binary or categorical feature.

`gridsearch` then tunes λ by GCV (generalized cross-validation), an in-sample shortcut to leave-one-out error — but still look at the curves and ask whether they are plausible.

Interpreting GAM Results: Partial Dependence Plots

Each panel is one $\hat{f}_j(x_j)$ with its confidence band, from Lab 3's electricity model:



Read the band too. Every day of the year appears four or five times, so it barely changes width; where data thin out it fans, just where readers want to extrapolate.

A sentence someone can act on: *"plan for mid-July demand about 11,400 MW above the late-April trough, a winter peak about 8,500 above it, and weekends about 3,000 below weekdays."* No straight line in day of year has two peaks.

Key Takeaways and Roadmap

GAMs: flexible, interpretable, and underutilized in business forecasting.

GAMs vs. Other Methods: When to Choose Each

Method	Nonlinear	Interactions	Interpretable	Use case
ARIMA / ETS	No	No	High	Univariate time series
Linear regression	No	Manual	High	Few predictors
GAM	Yes	Manual	High	Few predictors, explain effects
Random Forest	Yes	Yes	Medium	Many predictors, tabular
XGBoost	Yes	Yes	Low	Many predictors, accuracy first

The deciding column is usually **Interactions**. A GAM is additive by construction, so if the effect of price depends on whether it is a holiday week, you must add that interaction yourself (`te()`); a tree finds it unprompted.

Choose a GAM when you must explain each predictor's effect, the effects are curved, or a series has complex trend and seasonality (Prophet), and interpretability matters more than the last few points of accuracy. Whether a flexible model buys those points is an empirical question; Lectures 4–6 test it.

Lecture 3: Key Takeaways

1. **GAMs**, $y = \alpha + f_1(x_1) + \cdots + f_k(x_k) + \varepsilon$: nonlinear, yet additive, so each effect plots on its own.
2. The **smoothness penalty** prices bending; λ, chosen by GCV or cross-validation, trades fit against smoothness.
3. **Prophet** is a GAM for time series: trend + seasonality + holidays.
4. **pyGAM**: `s()` smooth, `l()` linear, `f()` categorical; `gridsearch` tunes λ.
5. **Partial dependence plots** turn the model into sentences a stakeholder can act on.

Next: Decision Trees — moving to tree-based methods that capture interactions automatically.

References I


Taylor, Sean J. and Benjamin Letham (2018). "Forecasting at Scale". In: *The American Statistician* 72.1, pp. 37–45.


Wood, Simon N. (2017). *Generalized Additive Models: An Introduction with R*. 2nd. Boca Raton, FL: Chapman and Hall/CRC.